Assignment 1 Report: Pentium M Hybrid Branch Predictor

Oct 7, 2026 · @Umar

Overview

I implemented a hybrid branch direction predictor modeled on the Pentium M, and it averages 5.713 mispredictions per 1,000 instructions (MPKI) across the 20 provided traces. That is lower than both the provided gshare example (6.306 MPKI) and a bimodal-only predictor (9.758 MPKI), and far below a static always-taken guess (86.312 MPKI).

The predictor combines a bimodal table indexed by branch address with a tagged, 2-way set-associative global predictor indexed by the branch address XOR the global history. Following the assignment, the loop predictor, branch target prediction and the PIR hash were not implemented. All code is in the `pm_predictor` class in `src/my_predictor.h`, which is the class `predict.cc` instantiates.

Design

The predictor has two parts, and the global part overrides the bimodal part whenever it holds an entry for the current branch, as in the Pentium M.

Bimodal predictor. A table of 2-bit saturating counters indexed by the low 15 bits of the branch address. Counter values 0–1 predict not taken and 2–3 predict taken. It is the gshare example's table with the history removed from the index.

Global predictor. A 2-way set-associative table of 2,048 rows (4,096 entries). The row is the low 11 bits of the branch address XOR an 11-bit global history register (GHR). Each entry holds a valid bit, an 8-bit tag taken from the low address bits, and a 2-bit counter. A lookup hits when either way in the row is valid and its tag matches.

Selection. On a global hit, the global entry's counter gives the prediction. On a miss, the bimodal prediction is used.

Allocation. A branch gets a global entry only when it misses in the global table and the bimodal predictor mispredicted it. This keeps the global table for branches the bimodal table cannot handle. A new entry's counter starts weakly toward the actual outcome (2 if taken, 1 if not).

Replacement. An invalid way is filled first. If both ways are valid, the least recently used way is replaced, tracked with one LRU bit per row.

Parameter	Value
Bimodal entries	32,768 (15-bit index)
Global rows × ways	2,048 × 2 = 4,096 entries
Global history length	11 bits
Tag size	8 bits
Counter size	2 bits
Replacement	LRU (1 bit per row)
Approximate storage	8 KB bimodal + 5.5 KB global entries + 256 B LRU ≈ 13.8 KB

The assignment allows a 2-way instead of 4-way global table and does not fix table sizes, so these sizes were chosen and then varied in the results below. The Pentium M's real tables are smaller.

Implementation

The simulator calls `predict()` for each branch, scores the guess, then calls `update()` with the real outcome. Only conditional branches are predicted; other branches return taken and do not touch any state.

`predict()` does the following:

1. Computes the bimodal index from the address and reads that counter's prediction.
2. Computes the global row (address XOR GHR) and the tag (low address bits).
3. Checks both ways of the row for a valid entry with a matching tag.
4. Returns the global counter's prediction on a hit, or the bimodal prediction on a miss.
5. Saves the bimodal index, global row, tag, hit way and bimodal prediction in `pm_update`, so `update()` can reuse them.

`update()` does the following:

1. Moves the bimodal counter toward the actual outcome, saturating at 0 and 3.

2. On a global hit, moves that entry's counter toward the outcome and marks the other way as least recently used.
3. On a global miss where bimodal mispredicted, allocates an entry in the row (invalid way first, otherwise the LRU way) and sets its tag and counter.
4. Shifts the actual outcome into the GHR.

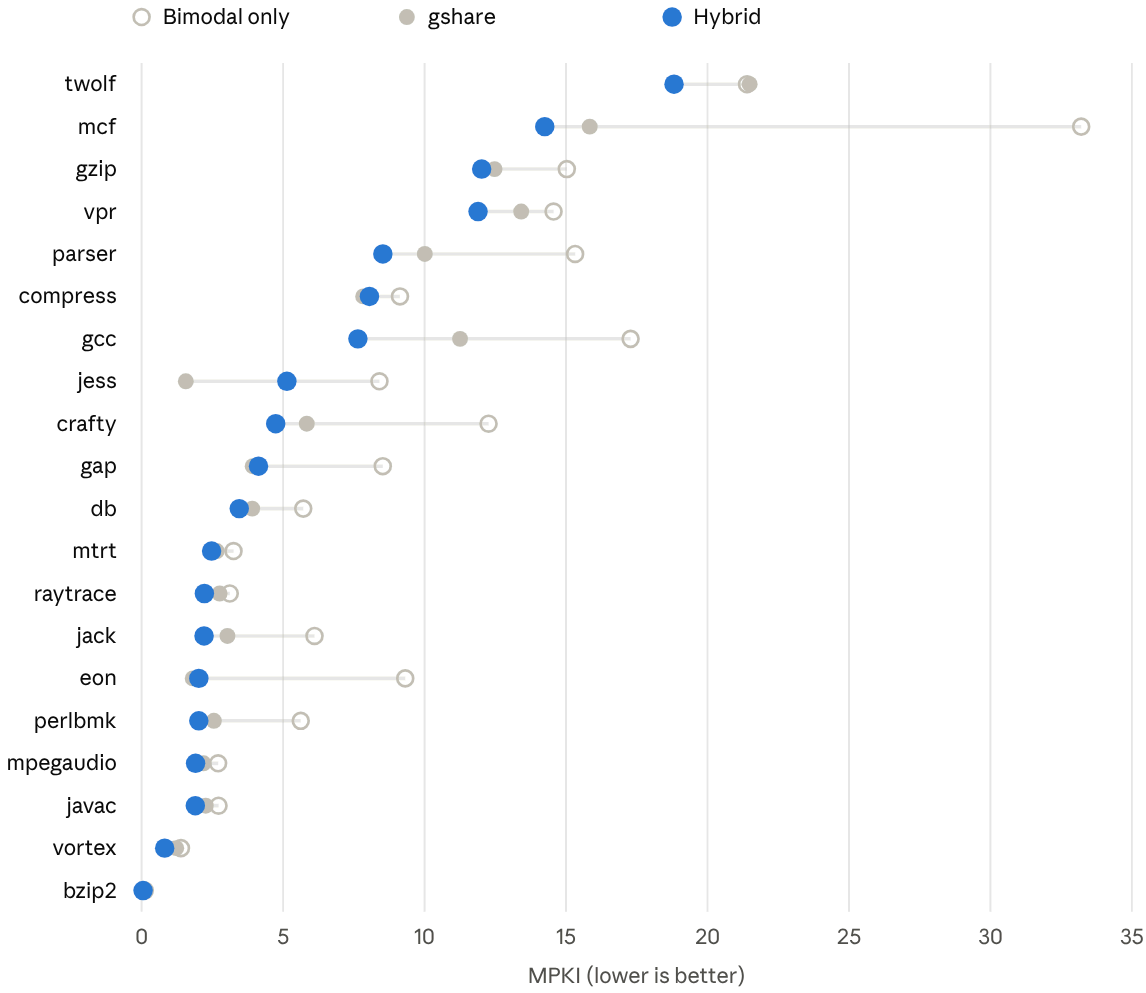
The bimodal table and counter logic reuse the provided gshare code, which the assignment permits. The tagged table is a small set-associative cache written directly in the class with a `global_entry` struct and a 2D array, rather than a separate cache class. Target prediction returns 0, since it was not required.

Results

The hybrid predictor averages 5.713 MPKI, against 6.306 for gshare, 9.758 for bimodal only, and 86.312 for the original always-taken stub. Each predictor was compiled into the same simulator and run on all 20 traces with `./run traces`.

Mispredictions per 1,000 instructions by trace

Hybrid beats gshare on 16 of 20 traces; average 5.71 vs 6.31 MPKI (bimodal only 9.76)



Measured with src/predict on the 20 CBP-2 traces; each trace is 100 million instructions

The always-taken stub is left off the chart because its values (24 to 178 MPKI) would flatten the other three.

Effect of global table size

With the bimodal table fixed at 32,768 entries and 2 ways per row, larger global tables lowered the average steadily.

Global rows	Tag bits	Average MPKI
1,024	6	6.074
2,048	6	5.799

Global rows	Tag bits	Average MPKI
2,048 (submitted)	8	5.713
4,096	8	5.506
8,192	8	5.140

History length was set equal to the number of index bits in each run.

Discussion

Adding the global predictor cut the bimodal-only average by 41%, from 9.758 to 5.713 MPKI. The biggest gains came on traces with branches that depend on recent history: gcc dropped from 17.282 to 7.644, mcf from 33.207 to 14.248, and crafty from 12.262 to 4.739. These are the branches the allocation rule sends to the global table, since bimodal keeps getting them wrong.

The hybrid beat gshare on 16 of 20 traces and on average (5.713 vs 6.306 MPKI). It lost on compress, jess, eon and gap, most clearly on jess (5.134 vs 1.562). gshare uses 32,768 counters with 15 bits of history for every branch, while the hybrid's global table has only 4,096 entries and 11 bits of history. Branches that need longer history or more global entries do better under gshare.

The comparison is not equal in storage. The hybrid uses about 13.8 KB against gshare's 8 KB, mostly because of the tags. Tags are the price of knowing when a global entry really belongs to the current branch, which removes the aliasing gshare suffers when two branches share a counter.

Larger global tables keep helping: going from 1,024 to 8,192 rows lowered the average from 6.074 to 5.140 MPKI. The real Pentium M kept its tables small because it was a mobile chip, where area and power matter as much as accuracy. Its global predictor targets only the branches bimodal misses, which is exactly what lets a small table work well.

The hardest traces are still mcf (14.248) and twolf (18.817). Their branches likely depend on data values rather than on branch history, so neither predictor captures them well.

Conclusion

A Pentium M–style hybrid of a bimodal table and a tagged 2-way global table reached 5.713 MPKI on average, beating gshare and the bimodal predictor alone. Letting the global table hold only the branches bimodal mispredicts made a modest 4,096-entry global table effective. Possible extensions are a 4-way global table, the loop predictor, and branch target prediction, which is the extra-credit `cpm_predictor`.

References

- Course assignment handout, “Assignment 1: Branch Predictor Implementation.”
- V. Uzelac and A. Milenković, “Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures,” ISPASS 2009.
- CBP-2 branch prediction infrastructure (cbp2-infrastructure-v2), including the gshare example in `my_predictor.h`, which the bimodal table reuses.
- Implementation assistance from Claude (Anthropic), an AI assistant, used as permitted by the course policy.